

SkinSense

Skin Analysis & Skincare Recommendation Platform
Project Proposal for UCS503P
Submitted to: Nisha Thakur
Thapar Institute of Engineering and Technology

Muskan Kohli, CSED* Himantveer Kaur, CSED[†] Ansh Bindal, CSED[‡]
Arnav Chandla, CSED[§]

August 20, 2026

1 Higher-order goal

This project aims to make personalized skincare guidance more accessible, reducing reliance on generic marketing-driven advice or costly dermatologist consultations for basic skin assessment. While the broader goal is accessible self-care, the immediate engineering objective is to translate a photo-based skin assessment into a measurable, deployable software solution that recommends suitable products and routines.

2 Time-to-value

- We will deliver meaningful increments quickly by getting the core pipeline (photo upload → classification → recommendation) working end-to-end early, so integration problems surface while there is still time to address them.
- We will prioritize fast feedback over perfection by using iterative development with CI-backed automation from the start, so every change is build-tested and deployable.
- Success is defined by what can be delivered and measured in the first iteration, then improved based on further testing.

3 Problem Statement

Most people do not have easy access to a dermatologist for basic skin assessment, and end up relying on subjective self-assessment or generic, marketing-driven product recommendations that do not account for their actual skin type, acne severity, or ingredient allergies. Common pain points include:

- **Subjective self-assessment:** people often misjudge their own skin type (oily/dry/normal) or the severity of their acne, leading to mismatched product choices.

*Roll No: 1024170453, Email: mkohli1_be24thapar.edu

[†]Roll No: 1024170271, Email: hkaur10_be24thapar.edu

[‡]Roll No: 1024170273, Email: abindal_be24thapar.edu

[§]Roll No: 1024170259, Email: achandla_be24thapar.edu

- **Generic recommendations:** most skincare advice available online is not personalized, and does not account for an individual’s specific skin condition.
- **Ingredient conflicts and allergies:** users have no easy way to check whether a recommended product contains an ingredient they should avoid.
- **No structured routine:** even when good products are chosen, users lack guidance on how to sequence them into a coherent morning/evening routine.

Why this matters now (contextual relevance): personalized skincare guidance is currently gated behind either paid consultations or trial-and-error, and a system that gives structured, explainable recommendations from a single photo can meaningfully lower that barrier.

4 Proposed Solution

4.1 Overview

Build a **web-based** skin analysis and skincare recommendation platform with the following components:

- **Photo upload and analysis:** users upload a face photo for skin type and acne severity assessment.
- **Skin type classification:** categorizes skin as oily, dry, or normal.
- **Acne severity classification:** a coarse-to-fine grading approach on the four-level Hayashi scale (mild/moderate/severe/very severe).
- **Rule-based recommendation engine:** matches skin type and acne severity against a curated product/ingredient database, with an allergy/ingredient exclusion filter and plain-language reasons for each recommendation.
- **Routine builder:** organizes recommended products into a morning/evening (AM/PM) routine.
- **Skin tone/undertone detection:** a lightweight, low-training-cost addition to refine recommendations (e.g., sunscreen suitability).

4.2 Core workflow

1. User uploads a face photo through the web interface.
2. Backend runs the skin type and acne severity classifiers and returns their outputs.
3. The recommendation engine matches these outputs, plus any allergy filters set by the user, against the product/ingredient database.
4. The system returns a ranked product list with reasons, organized into an AM/PM routine.

4.3 Operational constraints for an educational setting

- Keep the solution usable on low-to-mid range devices via responsive web design.
- Rely on established transfer-learning techniques, a published, benchmarked architecture for acne grading (Section 5.2), and deterministic rule-based logic elsewhere, avoiding unvalidated or ad hoc algorithms.
- Design for deployability using free-tier web hosting and managed databases.

5 Solution Approach

This is an engineering course project. The emphasis throughout is on correct, testable system design and integration, not on novel machine learning research.

5.1 Recommendation logic

The recommendation engine is rule-based, not machine learned:

- Match skin type, acne severity, skin tone, and allergy exclusions against tagged fields in the product/ingredient database.
- Apply deterministic matching rules sourced from the ingredient database’s documented suitability tags, not invented ad hoc.
- Always show the reason a product was recommended or excluded; never silently drop a result.

This keeps the decision core deterministic and fully unit-testable, which is deliberate: we have no reliable ground-truth label for “which product actually helped a given user,” so a learned recommender would not be justifiable at this stage.

5.2 Acne severity grading approach

Since ACNE04 has far more mild/moderate images than severe/very severe ones, a single 4-way classifier may struggle on the rarer classes. As a first attempt, we plan to try a two-step approach instead: a coarse low-vs-high severity split, followed by a finer classifier within each group. This is treated as an exploratory attempt rather than a guaranteed outcome; if it does not perform well in practice, a single classifier across all four levels serves as the fallback.

5.3 Web architecture

A typical 3-tier web architecture:

- **Frontend:** React, with a responsive upload page, results page, and a dark mode toggle.
- **Backend API:** FastAPI, handling image upload, model inference calls, and recommendation-engine queries.
- **Database:** PostgreSQL, storing the product catalog, ingredient data, and allergy/suitability tags, modeled with an explicit ER diagram.

5.4 CI/CD and fast delivery enabler

GitHub Actions will be used to:

- Automatically build and run backend unit tests and linting on every change.
- Deploy the application to a staging environment (Render/Railway) on successful builds.
- Enable safe, frequent releases of incremental features.

6 Evaluation Criterion

We define success using measurable metrics tied to the core classification and recommendation pipeline.

6.1 Primary evaluation metrics

- **Skin type classifier:** target $\geq 80\%$ accuracy on a held-out test set.
- **Acne severity classifier:** target $\geq 80\%$ accuracy, with a minimum acceptable floor of $\geq 70\%$, given the added difficulty of this task. Macro-F1 is reported alongside accuracy, since acne severity classes are naturally imbalanced (mild/moderate cases far outnumber severe/very severe), and accuracy alone can mask poor performance on the rarer classes.
- **Attribution:** both metrics are computed against the held-out test split of the respective training datasets, using a fixed, documented split before any tuning begins.

6.2 Secondary evaluation metrics

- **Recommendation engine test coverage:** target 100% coverage of rule branches, since the engine is deterministic and fully testable by design.
- **Inference time per scan:** measured and reported, to support a green-computing discussion of the lightweight backbone choice.
- **Allergy filter correctness:** verified via test cases that a flagged ingredient never appears in a returned recommendation.
- **User clarity:** pilot testers rate the clarity of the recommendation explanations on a 1–5 scale.

6.3 Pilot validation plan

- Once the system is functional end-to-end, recruit a small group of classmates/friends to test the upload-to-routine flow. No testers are lined up as of this proposal; this is a stated target for the testing phase.
- Collect a 1–5 clarity rating plus open feedback on the recommendations and routine output.

7 Scalability

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** the system should support growth in users and concurrent requests by:
 - decoupling frontend and backend components,
 - indexing common product/ingredient queries in PostgreSQL,
 - using a stateless API design.
2. **Operationally deployable (practically):** the system should run on common web hosting:
 - a managed PostgreSQL database,
 - platform-hosted deployment (Render/Railway),
 - a GitHub Actions CI/CD pipeline for staging.

8 Engine availability heuristic

A mature set of “engines” (tooling/services) is available to implement this as a web-based project:

- Pretrained CNN backbones for transfer learning, via PyTorch: MobileNetV2 for skin type, ResNet-50 for the coarse-to-fine acne severity grader.
- Free GPU training environments (Google Colab / Kaggle Notebooks).
- Web framework for REST APIs (FastAPI) with auto-generated documentation.
- Managed relational database (PostgreSQL).
- Test framework for unit/integration tests (`pytest`).
- CI runner and staging deployment target (GitHub Actions, Render/Railway).

These components are already mature and well documented, which lets us direct project effort toward model integration, rule design, correctness, and measurement.

9 Project scope and deliverables

9.1 Initial deliverable

- Photo upload interface
- Skin type classifier (oily/dry/normal) integrated behind a live API endpoint
- Acne severity classifier (4-level Hayashi grading; coarse-to-fine split attempted first, single classifier as fallback) integrated behind a live API endpoint
- Rule-based recommendation engine matching skin type and acne severity against the product database
- Basic frontend: upload page and results page
- CI: build + backend unit tests + linting
- CD to staging with a single pipeline job

9.2 Subsequent deliverables

- AM/PM routine builder
- Allergy/ingredient exclusion filter
- Skin tone/undertone detector
- “Why recommended” explanation tags for each product
- Lesion count estimation, decoupled from the severity classifier

9.3 Stretch goals

- A simple task queue (e.g., Celery) so a photo upload creates a job a worker processes asynchronously, with the frontend polling a status endpoint – a basic, working version of event-driven design, not a production-grade one.
- Flagging low-confidence scans using Monte Carlo Dropout, so users can be asked to retake an unclear photo.
- Checking accuracy across different skin tones (Fitzpatrick17k), and trying a simple fix like balanced sampling if a real gap shows up.
- Trying the skin-type model in-browser (e.g., TensorFlow.js) and comparing its speed to the server-based version.

A multi-class lesion detector is noted as a possible longer-term extension: fine-tune a pretrained object-detection model (e.g., YOLO, pretrained on COCO) on AcneSCU with moles, freckles, and melasma as categories separate from acne, so these are not misclassified as acne. This is not a committed deliverable, and is contingent on data availability.

10 Risks and mitigations

- **Acne severity accuracy:** expected to score lower than skin type; the coarse-to-fine attempt (Section 5.2) is a first try at improving this, with a plain classifier as fallback, and a safety-floor target of $\geq 70\%$ either way.
- **Moles/freckles misread as acne:** mitigated by including non-acne examples in validation to measure the error, with a fine-tuned detector (Section 9.3) as a longer-term fix.
- **Photo lighting/angle variability:** plan basic preprocessing (cropping, lighting normalization) before classification.
- **ACNE04 access:** request from the original authors; a lower-quality but accessible Kaggle dataset is the fallback.
- **Evaluation measurement ambiguity:** define accuracy/F1 targets and a fixed test split before development begins.
- **Operational issues in staging:** deploy early; rely on CI/CD for repeatable deployments.

11 Summary

This project proposes a deployable web platform that gives users a personalized, explainable skincare routine from a single uploaded photo. It meets the course criteria by emphasizing time-to-value through iterative delivery, using CI/CD to support frequent safe releases, and defining measurable evaluation criteria for both the classification models and the recommendation engine. The recommendation logic itself is kept deterministic and rule-based by design, keeping the project’s focus on correct, testable engineering.